

Delta Ingestion Framework - User Guide

In this article

- [Overview](#)
- [Steps to use DIF](#)
 - [1 - Prepare JSON config file based on the template](#)
 - [2 - Run Databricks_Job_Automation pipeline](#)
 - [3 - Validate dataset configuration in the ingestion_config table and the workflow](#)
 - [4 - Run test data ingestion](#)
- [Sample Databricks workflow](#)
 - [Parameters for Bronze task](#)
 - [Parameters for Standard task](#)
- [Ingestion_configuration table structure](#)
 - [field_config](#)
 - [mapping\(mandatory\)](#)
 - [{"op_type": "M"} or {"op_type": "M",active_flag":"N"}](#)
 - [This field allows multiple key-value pairs to be mapped. The key corresponds to a valid field in the source data, while the value represents the specific field value that signifies archival status.Note: The key is case insensitive; the value is case sensitive.](#)
 - [update_mode](#)
 - [The "all" designation means that DIF processes the entire incoming record.The "key" denotes that DIF only modifies two specific system flags in the standard table for archived records: 'adh_updated' and 'adh_archive_flag'.Note: "all" and "key" are case insensitive.](#)
- [Cluster Table Configuration \(Custom Config\)](#)
- [Data types in JSON schema](#)
- [Ingestion config templates](#)
- [Databricks runtime for DIF](#)
- [Reference Links](#)

Overview

- Delta Ingestion Framework (DIF) is a custom-built, configuration-based platform tool that facilitates data ingestion from heterogeneous data sources into Azure Databricks Delta Lake. The DIF can be used to load data into bronze and standard layers. It can be configured to ingest data from various sources such as Azure Data Lake Storage (ADLS) and Kafka topics. The supported data formats include CSV, Parquet, AVRO, ORC, and JSON. The overall structure of DIF consisting of a set of Python modules, and it can be divided into 2 major components, plus an ingestion config table.
- DIF follows a medallion architecture where raw data is first ingested into the bronze layer and then standardized into the standard layer. The standardization supported by DIF includes column renaming, data type conversion, de-duplication by primary keys, soft or hard deletes,

etc. More details on transformations can be found in the DIF architecture document. Users provide a JSON file to onboard a table for delta ingestion. The following [section](#) describes the steps involved in setting up delta ingestion.

Steps to use DIF

1 - Prepare JSON config file based on the template

The first step to onboard a new dataset is to prepare the dataset's configuration in JSON format. The template varies based on the source data format. Please refer to this [link](#) for reference.

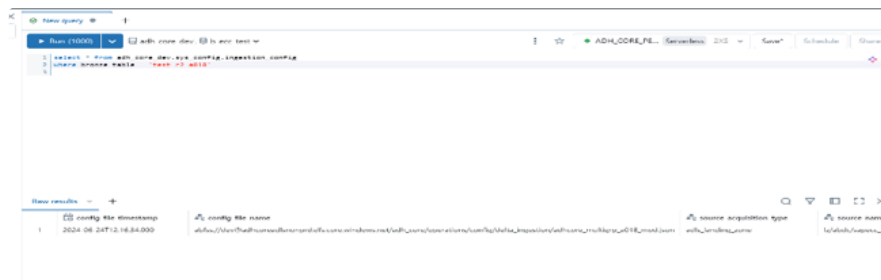
2 - Run Databricks_Job_Automation pipeline

Once the dataset's Json configuration is ready, the next step is to check in the configuration file and create a release in the repo. Once done, an instance of Databricks_Job_Automation pipeline will be automatically kicked off. If the job runs successfully, the dataset's configuration will be loaded into sys_config.ingestion_config table, and a Databricks workflow should be created in dev environment. The main tasks created by the pipeline are bronze and standard ingestion tasks. The detailed task entry and parameters are described [here](#). Please refer to this [guide](#) in details in terms of how to manage dataset configuration and run Databricks_Job_Automation.

3 - Validate dataset configuration in the ingestion_config table and the workflow

This step is to make sure the dataset configuration has been successfully loaded into sys_config.ingestion_config. The detailed field description of the table is described [here](#), and the below query can be used to validate the configuration in Databricks SQL Editor. After that, the Databricks workflow can be verified in Databricks Workflow blade.

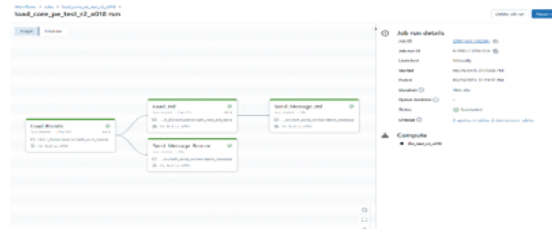
Select * from <catalog>.sys_config.ingestion_config where bronze_table='<configured table name>'



4 – Run test data ingestion

The last step to test whether the data ingestion works with the dataset's configuration is to prepare a test data, run the data ingestion, and validate the ingestion

- i. Prepare data to be ingested – Prepare or generate a data file for the dataset and put it to the dataset landing zone.
- ii. Run data ingestion job – Manually run the workflow to start the data ingestion.



- iii. Validate the bronze and standard tables – Once the workflow finishes, validate data in the tables with the data file.

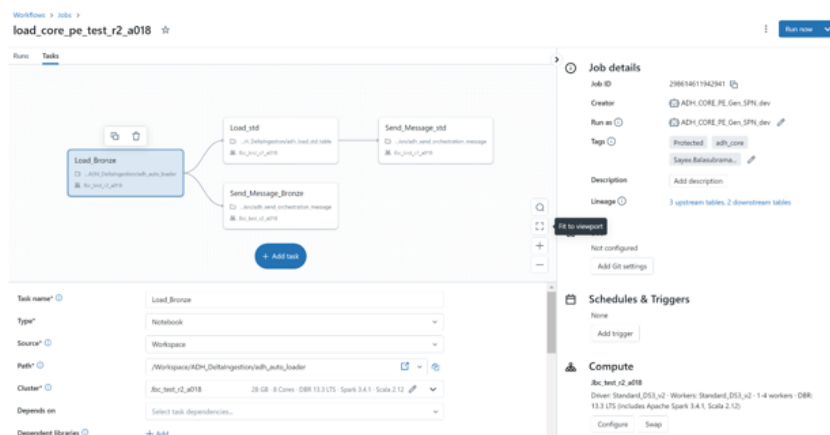
a. Sample data for bronze table

[illegible]

b. Sample data for standard table

[illegible]

Sample Databricks workflow



Parameters for Bronze task

- **Entry point:** ADH_DeltaIngestion/adh_auto_loader.py
- **Parameters:**
- **kv_scope:** Mandatory

This parameter specifies a Databricks key vault scope that is associated with the Azure Key Vault. The key vault is the place to hold all dynamic variables for the data ingestion

- **environment:** Mandatory

This parameter specifies a custodian for which the data ingestion is set up. This parameter is used to look for configured variables for the custodian in the key vault.

- **source_extract_type:** Optional

This parameter specifies the load type of the data ingestion. It can be 'I', meaning incremental load or 'F', meaning full load. If the parameter is not provided, DIF uses the value configured in the load_type column of ingestion_config table.

- **landing_source:** Optional

This parameter specifies the root path to the incoming data file on the blob storage. If the parameter is not provided, DIF uses the value configured in the source_name column of ingestion_config table.

- **catalog:** Mandatory

This parameter specifies the catalog the bronze table resides.

- **catalog_suffix:** Mandatory

This parameter specifies the catalog suffix such as 'dev'.

- **bronze_schema:** Mandatory

This parameter specifies the schema the bronze table resides.

- **bronze_table:** Mandatory

This parameter specifies the name of the bronze table.

- **process_id:** Optional

This parameter specifies the Databricks job id. The process id is one of the audit fields of the bronze table.

- **run_id:** Optional

This parameter specifies the assigned run id of the Databricks job at the runtime. The run_id is one of the audit fields of the bronze table.

- **stream_interval:** Optional

This parameter is only applicable to streaming datasets and specifies a time interval at which the micro batch is run for the streaming.

- **kafka_offset:** Optional

This parameter is only applicable to streaming datasets and specifies a Kafka offset at which the data ingestion should start.

Parameters for Standard task

- **Entry point:** ADH_DeltaIngestion/ adh_load_std_table.py

- **catalog:** Mandatory

This parameter specifies the catalog the standard table resides.

- **catalog_suffix:** Mandatory

This parameter specifies the catalog suffix such as 'dev'.

- **std_schema:** Mandatory

This parameter specifies the schema the standard table resides.

- **std_table:** Mandatory

This parameter specifies the name of the standard table.

- **process_id:** Optional

This parameter specifies the Databricks job id. The process id is one of the audit fields of the standard table.

- **run_id:** Optional

This parameter specifies the assigned run id of the Databricks job at the runtime. The run_id is one of the audit fields of the standard table.

Ingestion_configuration table structure

config_file_timestamp	timestamp
config_file_name	string
source_acquisition_type	string
source_name	string
source_data_format	string
source_data_format_options	string
source_data_metadata	string
load_type	string
unity_catalog	string
bronze_schema	string
bronze_table	string
std_schema	string
std_table	string
std_sql	string
std_column_map	string
std_primary_keys	string or list of dictionaries
std_orderby_keys	string
std_soft_del_flag	string
source_del_config	string
send_msg_flag	string

stream_interval	string
schedule	string
delta_properties	string
std_soft_del_data_upd_flag	string
std_schema_evolution_flag	string
bronze_schema_change_notification_flag	string
std_archive_config	string
custom_config	string

Field usage

- **config_file_timestamp:** DIF managed field

This field keeps a timestamp when the dataset is deployed.

- **config_file_name:** DIF managed field

This field keeps the dataset configuration file name

- **source_acquisition_type:** Mandatory

This field specifies the type of source. There are 3 types of data source DIF supports

adls_landing_zone:

This value indicates the source of data is landed in the landing zone on the Custodian's Storage account.

adls_kafka_sink:

This value indicates the source of data is created by Kafka connector and landed in the eapadlskafkasink<env>storage account

adh_catalog_volume:

This value indicates the source of data is landed in the Databricks Catalog Volumes.

eg. "source_name":

```
"sys_config/dif/Volumes_based_Ingestion_Implementation/csv_files"
```

kafka_stream:

This value indicates the source of data is Kafka stream.

stelo_replication:

This value indicates the source of data is stelo replication tool, and the bronze ingestion is not required.

- **source_name:** Mandatory

In the case of Landing Zone, Kafka Sink ADLS sources or DBK Catalog Volumes, this field will have the location of source data file. It could be Landing Zone folder, Kafka Sink folder or the databricks Catalog path.

In case of streaming data source from Kafka, this field specifies Kafka topic name information. The field supports '{queue_name}' in the name of the Kafka topic.

- **source_data_format:** Mandatory

This field specifies the incoming data format. The list of values to be used for this property is based on spark data formats. Following is the list of allowed values:

- i. csv (covers delimited text data and csv text data)
- ii. parquet
- iii. orc
- iv. avro
- v. json
- vi. delta. (Note: this indicates the source directly is written to the bronze table)

- **source_data_format_options:** Optional

This field requires a JSON string that includes the extended Autoloader options such as common autoloader options or the options related to the source data format.

e.g. 1: Common autoloader option: '{"cloudFiles.maxFilesPerTrigger": 10000}'

e.g. 2: CSV file option: '{"cloudFiles.maxFilesPerTrigger": 10000, "sep": ";", "quote": "^", "header": "true"}'

- **source_data_metadata:** Optional for Parquet file and Mandatory for CSV or JSON file

This field requires a JSON string that specifies the schema of incoming data source.

The format of the JSON schema needs to follow Spark's schema definition syntax. The available data types described [here](#).

e.g.: for a CSV file, the JSON schema look like this

```
1 {
2   fields": [
3     {
4       "metadata": {},
5       "name": "ID",
6       "type": "string",
7       "nullable": true
8     },
9     {
10      "metadata": {},
11      "name": "IBND",
12      "type": "string",
13      "nullable": true
14    },
15  ],
16  "type": "struct"
17 }
18 }
```

- **load_type:** Mandatory

This property specifies if source data is full load or incremental in nature. Following is the list of allowed values:

F: signifies full load; meaning incoming source data in bronze output table will be overwritten.

I: signifies incremental load; meaning incoming source data in bronze output table will be appended.

- **unity_catalog:** Mandatory

This field specifies the Unity Catalog name of the bronze and standard table. e.g.:
adh_core

- **bronze_schema:** Mandatory

This field specifies the catalog schema of the bronze table. e.g.: b_distribution

- **bronze_table:** Mandatory

This field specifies the table name of the bronze layer. e.g.: b_sapattp_zdt_sn_response

- **std_schema:** Mandatory

This field specifies the catalog schema of the standard layer. e.g.: s_distribution

- **std_table:** Mandatory

This field specifies the table name of the standard layer. e.g.:
s_sapattp_zdt_sn_response

- **std_sql:** DIF managed field

This field contains the SQL statement generated by SQL Generator. This field is the DIF utility managed field. As a requirement, when a new field is added to the bronze table, the new field will not be automatically added to the standard table. If the new field is reviewed and decide to be added to the standard table, this field needs to be empty out via the dataset deployment process.

- **std_column_map:** Optional

This field requires a JSON string and is used to map columns from the bronze table to columns in the standard table in supporting column renaming and data type transformations. DIF supports below data type transformations:

String to date/timestamp

```
1 # Syntax
2 {
3   "source_column": "<bronze column name>",
4   "target_column": "<Standard column name>",
5   "date or timestamp": "<time format string>"
6 }
```

 For **string to date/timestamp conversion** during **Bronze → Standard** processing, jobs must run on **Serverless compute** or **Job Compute 17.3 LTS or later (Spark 4.0.0)**. This is required to ensure **DIF compatibility across both Serverless and Job Compute**.

String/integral number to Boolean

```
1 # Syntax
2 {
3   "source_column": "<bronze column name>",
4   "target_column": "<Standard column name>",
5   "boolean": {"true": [colon separated value list], "false": [colon
6     separated value list] } #false is optional
7 }
```

String to int

```
1 # Syntax
2 {
3   "source_column": "<bronze column name>",
4   "target_column": "<Standard column name>",
5   "numeric":{"type":"int"}
6 }
```

String to number

```
1 # Syntax
2 {
3   "source_column": "<bronze column name>",
4   "target_column": "<Standard column name>",
5   "numeric":{"type":"decimal","precision": 9, "scale": 2} #type can
    be valid Spark data types, precision #and scale are optional and are
```

```

    required only #for decimal data type
6 }

```

The transformations are translated into SQL expressions and saved as part of SQL statement in std_sql column.

e.g.: below is a sample JSON string for the column mapping

```

1 {"field_map": [
2   { #---date transformation and column renaming---
3     "source_column": "ZZ_EXPIRATION_DT",
4     "target_column": "ZZ_EXPIRATION_DT_transformed",
5     "date": "yyyyMMDD"
6   },
7   { #---timestamp transformation and column renaming
8     ---
9     "source_column": "ZZ_UPDATE_TIMESTAMP",
10    "target_column": "ZZ_UPDATE_TIMESTAMP ",
11    "timestamp": "yyyyMMdd:HH:mm:ss.SSSSSSSS"
12  },
13  { #---boolean transformation and column renaming --
14    -
15    "source_column": "ZZ_EVT_TYP",
16    "target_column": "ZZ_EVT_TYP_transformed",
17    "boolean":{"true":["1","Y","yes","y"]}
18  },
19  { #---columns rename---
20    "source_column": "ZZ_INT_EXT",
21    "target_column": "ZZ_INT_EXT_transformed"
22  },
23  { #---numeric decimal transformation ---
24    "source_column": "old_field",
25    "target_column": "new_field",
26    "numeric":{"type":"decimal","precision": 9,
27    "scale": 2}
28  },
29  { #---numeric int transformation ---
30    "source_column": "old_field",
31    "target_column": "new_field",
32    "numeric":{"type":"int"}
33  }
34 ]
35 }

```

• std_primary_keys: Optional

This field specifies primary key columns in the standard table. The columns are used to apply dedupe and merge operations. e.g.: ZZ_GUID

There are two ways to configure the std_primary_keys through the ingestion config.

1. Column names as a comma separated string.

```

1 "std_primary_keys": "key1,key2,key3,key4"

```

The above string means that the four columns are treated as primary keys and used to apply dedupe and merge operations. As shown below.

Merge Condition: std.key1 = bronze.key1 and std.key2 = bronze.key2 and std.key3 = bronze.key3 and std.key4 = bronze.key4

2. Column names as a list of dictionaries (Allowing for "compare_mode" configuration)

Compare Mode decides which comparison operator to use while merging from bronze to standard table.

- "compare_mode": "null_safe" → means that the `<=>` (nulls safe operator) will be used
- No "compare_mode" → means, the `=` operator will be used.

```
1  "std_primary_keys": [  
2    {"column_name": "key1"},  
3    {"column_name": "key2"},  
4    {"column_name": "key3", "compare_mode": "null_safe"},  
5    {"column_name": "key4", "compare_mode": "null_safe"}  
6  ],
```

The above config means that the columns 'key1' and 'key2' will use `=` operator while merging to std, and the columns 'key3' and 'key4' will use the `<=>` nulls safe operator are treated as primary keys and used to apply dedupe and merge operations. As shown below.

Merge Condition: std.key1 = bronze.key1 and std.key2 = bronze.key2 and std.key3 <=> bronze.key3 and std.key4 <=> bronze.key4

⚠ This "std_primary_keys" config will not create actual table level primary key constraint. If the user wants to create a PK for, he standard table, it needs to be configured under the Delta_properties → "standard" → "primarykey".

Ful Delta properties json example present in Delta_properties Column description

To define a Primary Key in a standard table, the key must be specified in the Delta_properties column using JSON format. This ensures that the primary key is correctly recognized within the table schema.

```
1  "standard":  
2    {  
3      "primarykey": ["agr_num", "cond_record_num"]  
4    }
```

Ful Delta properties json example present in Delta_properties Column description

- **std_orderby_keys:** Optional

This field specifies columns in the standard table. The columns are used to apply dedupe and update operation. e.g.: for incremental dataset, DIF runs progressive comparison on the columns between the bronze and standard tables to update the standard table.

- **std_soft_del_flag:** Optional

This field should be used with the field 'source_del_config'. If the field is empty or 'N', DIF performs the hard deletion operation based on primary keys, order by keys, and source delete configuration. If the field contains 'Y', DIF performs soft deletion operation on the standard table.

- **std_soft_del_data_upd_flag:** Optional

This property will have the value of Y or N. This property is only applicable for Incremental Data and when soft delete is enabled via standard_soft_del_flag as mentioned above.

By default if this flag is not present in configuration or if this flag set to value of N, the framework will not update any of the data columns in Standard table during soft delete operation.

When this flag is set to Y, the framework will update all data columns along with soft delete related audit columns (adh_deleted & adh_delete_flag) in Standard table during soft delete operation.

It is recommended to use this flag (with a value of Y) when incoming CDC record for delete operation is a full record with latest values available for all data columns from the source system. Example :

```
1 | {  
2 |   "std_soft_del_data_upd_flag": "Y"  
3 | }
```

- **Source_del_config:** Optional

This field should be used with the field 'std_soft_del_flag'. For incremental datasets, this field accepts a Json string which contains columns and their values used to identify data rows to be deleted from the standard table. e.g: {"ZOPERATION": "D"}. This indicates that if the ZOPERATION column of data rows contains 'D', the data rows are either marked as soft deleted or physically deleted from standard table based on std_soft_del_flag configuration.

- **std_schema_evolution_flag** Optional

This configuration flag controls whether auto schema evolution is enabled for the Standard Layer during the Delta ingestion process

example config :

```
1 {  
2  
3   "std_schema_evolution_flag": "Y"  
4  
5 }
```

example value :

Value	Description
'Y'	Enables auto schema evolution.
'N'	Disables auto schema evolution.
null or any other value	Treated as 'N' ; schema evolution is disabled.

• **bronze_schema_change_notification_flag:** Optional

This configuration flag determines whether an email notification is triggered when a schema change is detected during the bronze ingestion process. When enabled, DIF alerts recipients via email, notifying them of the schema change and specifying the new columns added to the files (along with their file paths). The list of email recipients is retrieved from an Azure Key Vault secret, specifically the `<custodian-env>-email-recipients` secret.

To enable this feature, set the value to 'Y' or 'y'.

Note: This feature identifies schema change only for parquet files processed through `source_acquisition_type = 'adls_landing_zone'`

⚠ The data ingestion process for the bronze table remains unaffected by this flag, and any schema changes will still cause the bronze loading task to fail as it normally would. However, the subsequent run of the job will succeed. To ensure uninterrupted workflow execution, we recommend configuring the bronze load task with an auto-retry mechanism (i.e., run once and retry on failure). This setup allows the workflow jobs to run smoothly without interruption.

example config:

```
1 {  
2   "bronze_schema_change_notification_flag": "Y"
```

values:

Value	Description
'Y' or 'y'	Enables schema change email notification feature.
null or any other value	Disables schema change email notification feature.

The required Azure Key Vault Secrets:

<custodian>-email-recipients	Comma separated email addresses of the recipients example secret names CORE-PE-email-recipients or NHH-email-recipients example secret values example 1 first_recipient@xyz.com, second_recipient@xyz.com example 2 single_recipient@xyz.com
<custodian>-smtp-config	Json value with smtp details example secret names CORE-PE-smtp-config or NHH-smtp-config example value <pre> 1 { 2 "host": "smtp-us.ser.proofpoint.com", 3 "port": 587, 4 "sender": 5 "DoNotReply@adhdif.cencora.com", 6 "username": "ADHDelta-ING-Framework" </pre>

<custodian>-smtp-config-password	String value password
----------------------------------	-----------------------

Example Email Notification:



- **Send_msg_flag:** Optional

This field can accept either 'Y' or 'N'. it is used by the deployment process to determine whether the ingestion job needs to have tasks to send message.

- **Stream_interval:** Optional

This field specifies a time interval used by streaming datasets. Example values could be '5 minutes', '30 seconds'.

- **Delta_properties:** Optional

This field accepts a Json string that allows the user to provide various delta table properties as well as partition by or cluster by column list. E.g.:

```

1  "delta_properties":{
2    "bronze":
3      {
4        "properties":
5          {
6            "delta.deletedFileRetentionDuration": "interval 7 days",
7            "delta.logRetentionDuration": "interval 60 days"
8          },
9        "clusterby": ["knuma_ag", "knumh"],
10       "partitionby": ["knuma_ag", "knumh"],
11       "primarykey": ["knuma_ag", "knumh"]
12     },
13     "standard":
14       {
15         "properties":
16           {
17             "delta.deletedFileRetentionDuration": "interval 7 days",
18             "delta.logRetentionDuration": "interval 60 days",
19             "delta.enableChangeDataFeed": "true"
20           },
21         "clusterby": ["agr_num", "cond_record_num"],
22         "partitionby": ["agr_num", "cond_record_num"]
23         "primarykey": ["agr_num", "cond_record_num"]
24       }

```

⚠ partitionby and clusterby are mutually exclusive, we cannot have both properties on the same table. Please also refer to [the link](#) to determine which property best suit the table.

i For implementing Auto Liquid Clustering. use `"clusterby": "auto"`
(Auto Liquid Clustering is supported with DBR 15.4 LTS and above)

Sample Json config below (**Delta_properties**)

```
1  "delta_properties":{
2    "bronze":
3      {
4        "clusterby": "auto"
5      },
6    "standard":
7      {
8        "clusterby": "auto"
9      }
10 }
```

• **std_archive_config**: Optional

This field accepts a JSON string, enabling users to configure DIF's archival feature and specify the data archiving instructions. The JSON string accepts the below two key fields:

Field	Type	Sample Value	comment
field_config	mapping (mandatory)	{“op_type”: “M”} or {“op_type”: “M”,active_flag:”N”}	This field allows multiple key-value pairs to be mapped. The key corresponds to a valid field in the source data, while the value represents the specific field value that signifies archival status. Note: The key is case insensitive; the value is case sensitive.
update_mode	string (optional)	“all” or “key”	The “all” designation means that DIF processes the entire incoming record. The “key” denotes that DIF only modifies two specific system flags in the standard table for archived records: 'adh_updated' and 'adh_archive_flag'. Note: “all” and “key” are case insensitive.

Example configuration:

```
{“field_config”: {“op_type”: “M”}, “update_mode”: “key”}
```

⚠ Archival functionality requires specific setup: The load type should be incremental. The incremental dataset must have a primary key and archival flag configured correctly. If the archival configuration is incorrect, it may result in data corruption in the standard table..

- **custom_config:** Optional

This is a generic configuration column that accepts a JSON string to support extensible and feature-specific ingestion settings. The intent of this column is to avoid introducing new configuration columns for every new feature added to the DIF framework.

Currently, this field is used to support SAP cluster tables. For cluster tables, the JSON string can be used to specify the partial primary keys that identify the clustered data. Example:

```
1 { "cluster_table_partial_key": "cluster_pk1,cluster_pk2" }
2
3 OR
4
5 { "cluster_table_partial_key": "MANDT, BUKRS, BELNR, GJAHR" }
```

When configured, the ingestion framework uses the provided partial keys to correctly process and manage cluster table data.

 As this field is JSON-based, it can be extended in the future to support additional configurations, either in combination with the cluster table settings or independently, without requiring schema changes to the ingestion configuration.

Cluster Table Configuration (Custom Config)

 If `custom_config` contains `cluster_table_partial_key`, the ingestion is treated as a **cluster table**.

Important: Primary Key and Delete Config Requirements for Cluster Tables

For cluster tables, both **std_primary_keys** and **cluster_table_partial_key** (in `custom_config`) along with **Source_del_config** are mandatory.

The `std_primary_keys` must define the **full primary key of the table**, as it is required to uniquely identify records in the Standard table and is used during target lookup and merge execution.

The `cluster_table_partial_key` indicates only the subset of primary key columns that appear in delete records.

Providing both full and partial keys is **critical** for correct delete identification and merge behavior.

Cluster table configurations must be set up for either **soft delete** or **hard delete** (i.e ***Source_del_config*** is mandatory), because actual updates on cluster tables are performed as a delete followed by a re-insert.

Archival can be configured at the same time as soft or hard delete. To enable archival, follow the standard configuration steps. When the table is defined as a cluster table, archival processing will use the **partial key** instead of the full primary key.

Data types in JSON schema

Spark Data type:	DataType	- Type String used in Json schema file:data
Spark Data type:	NullType	- Type String used in Json schema file:void
Spark Data type:	AtomicType	- Type String used in Json schema file:atomic
Spark Data type:	NumericType	- Type String used in Json schema file:numeric
Spark Data type:	IntegralType	- Type String used in Json schema file:integral
Spark Data type:	FractionalType	- Type String used in Json schema file:fractional
Spark Data type:	StringType	- Type String used in Json schema file:string
Spark Data type:	CharType	- Type String used in Json schema file:char
Spark Data type:	VarcharType	- Type String used in Json schema file:varchar
Spark Data type:	BinaryType	- Type String used in Json schema file:binary
Spark Data type:	BooleanType	- Type String used in Json schema file:boolean
Spark Data type:	DateType	- Type String used in Json schema file:date
Spark Data type:	TimestampType	- Type String used in Json schema file:timestamp
Spark Data type:	TimestampNTZType	- Type String used in Json schema file:timestamp_ntz
Spark Data type:	DecimalType	- Type String used in Json schema file:decimal
Spark Data type:	DoubleType	- Type String used in Json schema file:double
Spark Data type:	FloatType	- Type String used in Json schema file:float
Spark Data type:	ByteType	- Type String used in Json schema file:byte
Spark Data type:	IntegerType	- Type String used in Json schema file:integer
Spark Data type:	LongType	- Type String used in Json schema file:long
Spark Data type:	ShortType	- Type String used in Json schema file:short
Spark Data type:	AnsiIntervalType	- Type String used in Json schema file:ansiinterval
Spark Data type:	DayTimeIntervalType	- Type String used in Json schema file:daytimeinterval
Spark Data type:	YearMonthIntervalType	- Type String used in Json schema file:yearmonthinterval
Spark Data type:	CalendarIntervalType	- Type String used in Json schema file:interval
Spark Data type:	ArrayType	- Type String used in Json schema file:array
Spark Data type:	MapType	- Type String used in Json schema file:map
Spark Data type:	StructType	- Type String used in Json schema file:struct
Spark Data type:	VariantType	- Type String used in Json schema file:variant
Spark Data type:	UserDefinedType	- Type String used in Json schema file:userdefinedtype

Ingestion config templates

1. Template for batch ingestion

a. Template for batch CSV file

```
1 {
2   "source_acquisition_type": "adls_landing_zone",
3   "bronze_schema": "b_ecc",
4   "bronze_table": "A030",
5   "load_type": "f",
6   "source_name": "lz/abdc/sapecc_a030",
7   "source_data_format": "csv",
8   "source_data_format_options": {
9     "header": false,
```

```

10     "quote": "^",
11     "sep": "|"
12 },
13 "source_data_metadata": {
14     "fields": [{
15         "metadata": {},
16         "name": "KAPPL",
17         "type": "string",
18         "nullable": false
19     }, {
20         "metadata": {},
21         "name": "KSCHL",
22         "type": "string",
23         "nullable": false
24     }, {
25         "metadata": {},
26         "name": "VKORG",
27         "type": "string",
28         "nullable": false
29     }, {
30         "metadata": {},
31         "name": "VTWEG",
32         "type": "string",
33         "nullable": false
34     }, {
35         "metadata": {},
36         "name": "KUNNR",
37         "type": "string",
38         "nullable": false
39     }, {
40         "metadata": {},
41         "name": "KONDM",
42         "type": "string",
43         "nullable": false
44     }, {
45         "metadata": {},
46         "name": "DATBI",
47         "type": "string",
48         "nullable": false
49     }, {
50         "metadata": {},
51         "name": "DATAB",
52         "type": "string",
53         "nullable": true
54     }, {
55         "metadata": {},
56         "name": "KNUMH",
57         "type": "string",
58         "nullable": true
59     }
60     ],
61     "type": "struct"
62 },
63 "unity_catalog": "adh_hhd",
64 "std_schema": "std_ecc",
65 "std_table": "cust_mat_prc_grp_a030",
66 "std_primary_keys":
"application,cond_type,sales_org,dist_channel,cust_nbr,mat_prc_grp,co
nd_valid_to_date",
67 "std_orderby_keys": "adh_data_object_timestamp",
68 "std_sql": null,
69 "std_column_map": {
70     "field_map": [{
71         "source_column": "KAPPL",
72         "target_column": "application"
73     }, {
74         "source_column": "KSCHL",

```

```

75         "target_column": "cond_type"
76     }, {
77         "source_column": "VKORG",
78         "target_column": "sales_org"
79     }, {
80         "source_column": "VTWEG",
81         "target_column": "dist_channel"
82     }, {
83         "source_column": "KUNNR",
84         "target_column": "cust_nbr"
85     }, {
86         "source_column": "KONDM",
87         "target_column": "mat_prc_grp"
88     }, {
89         "source_column": "DATBI",
90         "target_column": "cond_valid_to_date",
91         "date": "M/d/y"
92     }, {
93         "source_column": "DATAB",
94         "target_column": "cond_valid_from_date",
95         "date": "M/d/y"
96     }, {
97         "source_column": "KNUMH",
98         "target_column": "cond_rec_nbr",
99         "numeric": {"type": "decimal", "precision": 9,
100     "scale": 2}
101     }
102 ],
103     "delta_properties": {
104         "bronze": {
105             "properties": {
106                 "delta.deletedFileRetentionDuration": "interval 30
107 days",
108                 "delta.logRetentionDuration": "interval 30 days"
109             }
110         },
111         "standard": {
112             "properties": {
113                 "delta.deletedFileRetentionDuration": "interval 30
114 days",
115                 "delta.logRetentionDuration": "interval 30 days",
116                 "delta.enableChangeDataFeed": "true"
117             }
118         },
119         "std_soft_del_data_upd_flag": "Y",
120         "std_schema_evolution_flag": "Y",
121         "bronze_schema_change_notification_flag": "Y",
122         "std_archive_config": {"field_config": {"op": "M"}, "update_mode":
123     "key" }
124     }

```

Template for batch Parquet file

```

1  {
2      "source_acquisition_type": "adls_landing_zone",
3      "source_name": "lz/abdc/sapbw_bv_emanifest_data",
4      "source_data_format": "parquet",
5      "source_data_format_options": null,
6      "source_data_metadata": null,
7      "load_type": "I",
8      "unity_catalog": "adh_hhd",
9      "bronze_schema": "b_bwhana",
10     "bronze_table": "emanifest_data",
11     "schedule": null,

```

```

12     "std_schema": "std_bwhana",
13     "std_table": "emanifest_data",
14     "std_primary_keys": "load_nbr,stop_nbr_h,product_id",
15     "std_orderby_keys": "adh_data_object_timestamp",
16     "std_column_map": {
17         "field_map": [{
18             "source_column": "load_nbr",
19             "target_column": "load_nbr"
20         }, {
21             "source_column": "stop_nbr_h",
22             "target_column": "stop_nbr_h"
23         }, {
24             "source_column": "signature_group",
25             "target_column": "signature_group"
26         }, {
27             "source_column": "sign_req_h",
28             "target_column": "sign_req_h"
29         }, {
30             "source_column": "stop_detail_seq",
31             "target_column": "stop_detail_seq",
32             "numeric": {"type": "int"}
33         }, {
34             "source_column": "DATAB",
35             "target_column": "cond_valid_from_date",
36             "date": "M/d/y"
37         }
38     ]
39 },
40 "std_soft_del_data_upd_flag": "Y",
41 "std_schema_evolution_flag": "Y",
42 "bronze_schema_change_notification_flag": "Y",
43 "std_archive_config": {"field_config": {"op": "M"}, "update_mode":
44 "key" }
45 }

```

Template for batch JSON format

```

1  {
2      "source_acquisition_type": "adls_landing_zone",
3      "source_name": "lz/abdc/ecc",
4      "source_data_format": "json",
5      "source_data_format_options": null,
6      "source_data_metadata": '{
7          "fields": [{
8              "metadata": {},
9              "name": "ordertime",
10             "type": "long",
11             "nullable": false
12         }, {
13             "metadata": {},
14             "name": "orderid",
15             "type": "integer",
16             "nullable": false
17         }, {
18             "metadata": {},
19             "name": "itemid",
20             "type": "string",
21             "nullable": false
22         }, {
23             "metadata": {},
24             "name": "address",
25             "type": "string",
26             "nullable": false
27         }
28     ],
29     "type": "struct"

```

```

30         },
31         "load_type": "I",
32         "unity_catalog": "adh_hhd",
33         "bronze_schema": "b_distribution",
34         "bronze_table": "b_item",
35         "schedule": null,
36         "std_schema": "s_distribution",
37         "std_table": "s_item",
38         "std_primary_keys": "load_nbr,stop_nbr_h,product_id",
39         "std_orderby_keys": "adh_data_object_timestamp",
40         "std_column_map": {
41             "field_map": [{
42                 "source_column": "load_nbr",
43                 "target_column": "load_nbr"
44             }, {
45                 "source_column": "stop_nbr_h",
46                 "target_column": "stop_nbr_h"
47             }, {
48                 "source_column": "signature_group",
49                 "target_column": "signature_group"
50             }, {
51                 "source_column": "sign_req_h",
52                 "target_column": "sign_req_h"
53             }, {
54                 "source_column": "stop_detail_seq",
55                 "target_column": "stop_detail_seq",
56                 "numeric": {"type": "int"}
57             }, {
58                 "source_column": "DATAB",
59                 "target_column": "cond_valid_from_date",
60                 "date": "M/d/y"
61             }
62         ]
63     }
64 }

```

1. Template for streaming ingestion

a. Template for Kafka AVRO payload

```

1  {
2      "source_acquisition_type": "kafka_stream",
3      "source_name": "{queue_env}.ATTP.SAPSR3.ZDT_SN_REQ",
4      "source_data_format": "avro",
5      "source_data_format_options": null,
6      "source_data_metadata": null,
7      "load_type": "I",
8      "unity_catalog": "adh_hhd",
9      "bronze_schema": "b_distribution",
10     "bronze_table": "b_zdt_sn_req",
11     "schedule": null,
12     "std_schema": "s_distribution",
13     "std_table": "s_zdt_sn_req",
14     "std_primary_keys": "load_nbr,stop_nbr_h,product_id",
15     "std_orderby_keys": "adh_data_object_timestamp",
16     "std_column_map": {
17         "field_map": [{
18             "source_column": "load_nbr",
19             "target_column": "load_nbr"
20         }, {
21             "source_column": "stop_nbr_h",
22             "target_column": "stop_nbr_h"
23         }, {
24             "source_column": "signature_group",
25             "target_column": "signature_group"
26         }, {

```

```

27         "source_column": "sign_req_h",
28         "target_column": "sign_req_h"
29     }, {
30         "source_column": "stop_detail_seq",
31         "target_column": "stop_detail_seq",
32         "numeric": {"type": "int"}
33     }, {
34         "source_column": "DATAB",
35         "target_column": "cond_valid_from_date",
36         "date": "M/d/y"
37     }
38 ]
39 }
40 }

```

Template for Kafka dataset with JSON format

```

1 {
2     "source_acquisition_type": "kafka_stream",
3     "source_name": "{queue_env}.Test.json.databricks",
4     "source_data_format": "json",
5     "source_data_format_options": null,
6     "source_data_metadata": '{
7         "fields": [{
8             "metadata": {},
9             "name": "ordertime",
10            "type": "long",
11            "nullable": false
12        }, {
13            "metadata": {},
14            "name": "orderid",
15            "type": "integer",
16            "nullable": false
17        }, {
18            "metadata": {},
19            "name": "itemid",
20            "type": "string",
21            "nullable": false
22        }, {
23            "metadata": {},
24            "name": "address",
25            "type": "string",
26            "nullable": false
27        }
28    ],
29     "type": "struct"
30     },
31     "load_type": "I",
32     "unity_catalog": "adh_hhd",
33     "bronze_schema": "b_distribution",
34     "bronze_table": "b_item",
35     "schedule": null,
36     "std_schema": "s_distribution",
37     "std_table": "s_item",
38     "std_primary_keys": "load_nbr,stop_nbr_h,product_id",
39     "std_orderby_keys": "adh_data_object_timestamp",
40     "std_column_map": {
41         "field_map": [{
42             "source_column": "load_nbr",
43             "target_column": "load_nbr"
44         }, {
45             "source_column": "stop_nbr_h",
46             "target_column": "stop_nbr_h"
47         }, {
48             "source_column": "signature_group",
49             "target_column": "signature_group"

```

```

50     }, {
51         "source_column": "sign_req_h",
52         "target_column": "sign_req_h"
53     }, {
54         "source_column": "stop_detail_seq",
55         "target_column": "stop_detail_seq",
56         "numeric": {"type": "int"}
57     }, {
58         "source_column": "DATAB",
59         "target_column": "cond_valid_from_date",
60         "date": "M/d/y"
61     }
62 ]
63 }
64 }

```

Databricks runtime for DIF

The current recommended DBR is 13.3 LTS. However, with DBR 13.3 LTS, the row level concurrency is not supported by default. The user might run into the job failure when running multiple first-time std ingestions concurrently.

DBR 14.* with shared access mode has a bug in the delta API used in DIF, and thus it is not compatible with DIF.

DBR 15.* with shared access mode has an issue in the delta API used in DIF, and thus it is not compatible with DIF.

Reference Links

[Databricks - Data Ingestion Architecture, Design, and Implementation](#)

[Databricks - Bronze Ingestion Operational Runbook](#)

[Databricks - Standard Ingestion Operational Runbook](#)